

دانشگاه تهران

پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر

تمرین شماره: ۲

شبکه‌های عصبی و یادگیری عمیق

نام و نام خانوادگی: آرتین توسلی

شماره دانشجویی: ۸۱۰۱۰۲۵۴۳

نیم‌سال دوم

سال تحصیلی ۴۰۴-۴۰۵

فهرست مطالب

۴	۱ تشخیص چهره با استفاده از تابع زیان فاصله بین کلاسی
۴	۱.۱ تئوری
۴	۱.۱.۱ تابع زیان طبقه بندی کلاسیک
۷	۲.۱.۱ توابع زیان و فاصله بین کلاسی
۹	۳.۱.۱ ارتباط فاصله زاویه ای به تشخیص چهره
۱۱	۴.۱.۱ تابع زیان IAM
۱۶	۵.۱.۱ دیگر توابع زیان
۲۰	۲.۱ پیاده سازی
۲۰	۱.۲.۱ آماده سازی داده
۲۴	۲.۲.۱ مدل پایه CNN با Softmax
۲۷	۳.۲.۱ پیاده سازی Angular Margin Loss
۳۱	۴.۲.۱ مقایسه نهایی، تحلیل و ارتباط با مقاله IAM
۳۵	کتابنامه

فهرست تصاویر

۱۰.۱	توزیع مقادیر پیکسل‌ها برای یک نمونه تصادفی از هر یک از ۱۰ کلاس	۲۰
۲۰.۱	نتایج عملی ماتریس کوواریانس	۲۲
۳۰.۱	Confusion Matrix طبقه‌بند میانگین	۲۳
۴۰.۱	نمودارهای تغییرات تابع زیان و دقت برای مجموعه‌های آموزش و تست در طول ۳۰ اپیاک، به تفکیک اندازه‌های دسته متفاوت.	۲۴
۵۰.۱	مقایسه Confusion Matrix مدل CNN روی داده‌های آموزش و تست.	۲۵
۶۰.۱	نگاشت دوبعدی فضای ویژگی با استفاده از الگوریتم PCA برای اندازه‌های دسته مختلف.	۲۶
۷۰.۱	نقشه حرارتی زاویه به درجه بین مراکز کلاس‌ها در فضای ویژگی برای داده‌های آموزش و تست.	۲۷
۸۰.۱	نمودارهای تغییرات تابع زیان و دقت برای مجموعه‌های آموزش و تست در طول ۳۰ اپیاک با استفاده از تابع Angular Margin.	۲۸
۹۰.۱	ماتریس درهم‌ریختگی مدل Angular Margin روی مجموعه‌های آموزش و تست.	۲۹
۱۰۰.۱	نگاشت دوبعدی فضای ویژگی مدل Angular Margin.	۳۰
۱۱۰.۱	نقشه حرارتی زاویه به درجه بین مراکز کلاس‌ها در فضای ویژگی نرمال شده.	۳۰

فهرست جداول

۱۰۱ مقایسه عملکرد و متریک‌های فضای ویژگی بین مدل پایه و مدل مبتنی بر حاشیه زاویه‌ای ۳۱

فصل ۱

تشخیص چهره با استفاده از تابع زیان فاصله بین کلاسی

۱.۱ تئوری

۱.۱.۱ تابع زیان طبقه بندی کلاسیک

Softmax تابع زیان

تابع زیان Softmax به صورت زیر تعریف می‌شود:

$$L_{soft} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{w_{y_i}^T x_i}}{\sum_{j=1}^C e^{w_j^T x_i}} \quad (1.1)$$

اجزای فرمول:

• N : تعداد نمونه‌ها در یک Batch .

- C : تعداد کل کلاس ها .
- x_i : بردار ویژگی استخراج شده از لایه یکی مانده به آخر برای نمونه i -ام .
- y_i : کلاس هدف نمونه i .
- w_{y_i} : بردار وزن مربوط به کلاس هدف نمونه i .

این تابع در اصل منفی احتمال کل داده ها را سعی میکند کمینه کند (یعنی احتمال درست طبقه بندی کردن کل داده ها را بیشینه کند). در این تابع اول امتیاز هر نمونه محاسبه میشود یعنی خروجی شبکه $(w^T x)$ به ازای اون نمونه (Logits) و حال این اعداد چون از جنس امتیاز هستند و نه احتمال، نرمالایز اش میکند (مخرج کسر در معادله)، حال با توجه به قانون زیر،

$$P(Data) = P(y_1|x_1)P(y_2|x_2)...P(y_N|x_N) \quad (2.1)$$

احتمال کل داده ها محاسبه میشود ولی چون کار با ضرب دشوار هست و مشکلات عددی ممکن است درست کند (ضرب چندین عبارت ریز از دقت type data ای که داریم نگهداری خارج بشود)، لگاریتم میگیریم و ضرب ها تبدیل به حاصل جمع میشود.

$$P(Data) = \sum_{i=1}^N \log P(y_i|x_i) \quad (3.1)$$

مدل زمانی یک نمونه را به کلاس درست (y_i) نسبت می دهد که مقدار امتیاز آن با بردار وزن کلاس خودش، از امتیاز آن با سایر کلاس ها بیشتر باشد $(w_j^T x_i > w_{y_i}^T x_i)$ برای هر $y_i \neq j$.

این امتیاز ها که اساس این تابع میباشد را اگر باز کنیم،

$$w_j^T x_i = ||w_j|| ||x_i|| \cos \theta_{i,j} \quad (4.1)$$

طبقه بندی بر اساس زاویه (θ) بین ویژگی x_i و وزن کلاس w_j انجام می شود. مدل سعی می کند زاویه با کلاس درست را به حداقل برساند تا مقدار کسینوس (و در نتیجه احتمال) بیشینه شود .

تابع Softmax به تنهایی تضمین کننده فاصله زیاد بین کلاس ها نیست. تابع Softmax، صرفا به دنبال یافتن مرز تصمیمی برای حداقل کردن خطای دسته بندی است. مشکل اساسی این تابع این است که به محض قرار گرفتن داده ها در سمت درست مرز تصمیم (یعنی زمانی که حاصل ضرب داخلی کلاس صحیح کمی از

سایر کلاس‌ها بیشتر شود)، مقدار زیان به شدت افت می‌کند. در این نقطه، گرادیان‌های تولید شده برای آموزش شبکه بسیار کوچک می‌شوند و بهینه‌سازی فاصله عملاً متوقف می‌گردد. در نتیجه، تابع Softmax به صورت ذاتی فاقد هرگونه مکانیزم مشخص (مانند تعریف یک margin) است. این تابع هیچ اجباری برای شبکه ایجاد نمی‌کند که پس از دسته‌بندی صحیح داده‌ها، نمونه‌های درون یک کلاس را بیشتر فشرده کند یا فاصله بین خوشه‌های کلاس‌های مختلف را افزایش دهد.

تابع زیان Scale

تابع زیان Scale همان تابع Softmax می‌باشد با این تفاوت که در آن بردار ویژگی و بردار وزن نرمال‌سازی شده ($1 = \|w\|$ و $1 = \|x\|$) تا خروجی شبکه (Logits) فقط به زاویه بستگی داشته باشد ($1 \cdot 1 \cdot \cos \theta$) و همین‌طور خروجی در یک عدد ثابت ضرب می‌شود.

$$L_{scale} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s \cos \theta_{i,y_i}}}{\sum_{j=1}^C e^{s \cos \theta_{i,j}}} \quad (5.1)$$

ضریب s : از آنجا که مقدار کسینوس بین -1 و 1 است، این بازه برای تابع نمایی (e^x) بسیار کوچک است (حداکثر 71.2 و حداقل 37.0) و این ضریب را ضرب می‌کنیم تا دامنه logits ها بیشتر بشود، ضریب s معمولاً بین 30 تا 64 قرار داده می‌شود. تأثیرات این امر:

- قطعیت مدل بیشتر می‌شود، یعنی confidence مدل بیشتر می‌شود. به جای اینکه e به توان یک عدد خیلی کوچک برسونه (که یعنی تفاوت احتمالش با کلاس‌های دیگه بسیار کم می‌باشند هر چند هنوز بیشینه هست)، به توان یک عدد خیلی بزرگ میرسد برای کلاس درست و احتمال کلاس درست نزدیک 1 و احتمال کلاس غلط نزدیک 0 می‌شود.
- بهبود همگرایی، بدون این ضریب گرادیان‌های شبکه بسیار کوچک خواهد بود و یعنی عملاً بخاطر اینکه confidence کمی دارد مدل، آموزش هم سخت می‌شود.
- این ضریب را ثابت نگه می‌داریم تا شبکه فقط به تغییر دادن زاویه تمرکز کند.

دلایل برتری این تابع به تابع Softmax:

- با حذف اثر طول بردارها ($\|x\|$ و $\|w\|$)، شبکه مجبور می‌شود فقط روی جهت بردارها تمرکز کند که در بازشناسی چهره اهمیت حیاتی دارد (کیفیت و یا شدت نور عکس در طول این بردار ها تاثیر دارد اما در این تسک طبقه بندی برای ما این موارد اهمیتی ندارد چون می‌خواهیم عکس اقای x چه با کیفیت

- و چه بی کیفیت و چه روشن و چه تاریک در یک جا باشد و فاصله درون کلاسی اش بسیار کم باشد).
- بهبود همگرایی: همانطور که بالاتر گفته شد، ضریب s باعث می شود گرادیان ها در فرآیند Back-propagation به اندازه کافی بزرگ باشند تا شبکه بتواند به سرعت یاد بگیرد. بدون s ، تفاوت بین کلاس ها در خروجی Softmax بسیار ناچیز می شد. (فرآیند آموزش را روی یک Hypersphere پایدار می سازد)

مشکلاتی که نمی تواند حل کند:

- نبود Margin در مرز تصمیم گیری: مرز بین دو کلاس $\cos \theta_1 = \cos \theta_2$ این یعنی مثل توضیحات بالا مدل اگر 1% مطمئن باشد درست تشخیص داده تابع هزینه به شدت کاهش پیدا میکند و این مرز robust نیست چون کوچکترین نویز میتواند باعث طبقه بندی اشتباه بشود. در توابعی مثل ArcFace، مرز به $\cos(\theta_1 + m) = \cos \theta_2$ تغییر می کند که مدل را مجبور می کند یک فاصله بین کلاس ها ایجاد کند، اما L_{scale} فاقد این قدرت است.
- جفت کلاس های سخت: اگر دو کلاستر به هم خیلی شبیه باشند (برای مثال در تشخیص چهره، شخص x نسبت به y شباهت زیادی دارد مثل چشم و گوش)، در این حالت ها تابع هزینه منطقاً باید سعی کند این دو تا را از هم دور نگه دارد تا اشتباه طبقه بندی نشود ولی در این تابع فقط یک میانگین کلی از شباهت ها را می بیند.
- دو مورد بالا باعث میشوند که واریانس برون کلاسی گسترش نیابد.
- کم نکردن فاصله درون کلاسی: این تابع زاویه را تا حدی کاهش میدهد که درست طبقه بندی بشود و بیشتر از آن فشاری نمیدهد که این زاویه کوچکتر بشود و یعنی Cluster های مترکم نمیسازد و یعنی ویژگی های یک فرد روی Hypersphere پخش میشود.

در واقع فقط از فضای اقلیدسی به کروی رفتیم و هنوز مشکلات زیادی در این تابع وجود دارد.

۲.۱.۱ توابع زیان و فاصله بین کلاسی

در مقاله از Inter-class Variance نام برده شده است. همانطور که میدانیم واریانس مربع فاصله ها از میانگین میباشد اما در اینجا کمی توصیفمان فرق دارد. در واقع الهام گرفته شده از مفهوم واریانس، ما فاصله زاویه ای مرکز کلاس (بردار وزن کلاس) را با نمونه ها حساب میکنیم. ابتدا بیان میکنیم چرا مرکز کلاس تقریباً

همان بردار وزن کلاس میشود در نظر گرفت. تابع هزینه Scale به صورت زیر بود:

$$L_{scale} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s \cos \theta_{i,y_i}}}{\sum_{j=1}^C e^{s \cos \theta_{i,j}}} \quad (6.1)$$

حال در صورت این کسر، زاویه θ_{i,y_i} آمده است که به معنای فاصله زاویه ای بین بردار w_{y_i} و x_i میباشد. این فرمول به دنبال بیشینه کردن این کسینوس زاویه هست، یعنی خود زاویه را کمینه بکند یعنی هر نمونه میخواهد زاویه اش را با بردار وزن کلاس حقیقی اش کمینه کند پس به نحوی با برآیند گیری برای هر نمونه، بردار وزن هر کلاس مرکز آن کلاس میباشد. واریانس برون کلاسی پس به ما نشان می دهد که مرکز کلاس های غیر خودی (تمام کلاس ها جز کلاس حقیقی آن نمونه) از بردارهای ویژگی نمونه ما چقدر دور میباشد. ما تمایل داریم این فاصله زیاد بشود ولی اول بیاید ببینیم این مقاله با معرفی تابع هزینه L_{IAM} چطور واریانس برون کلاسی را وارد فرمول L_{scale} میکند.

$$L_{IAM} = \frac{1}{N} \sum_{i=1}^N \log \frac{\frac{1}{C-1} \sum_{j=1, j \neq y_i}^C e^{s \cos \theta_{i,j}}}{\sum_{j=1}^C e^{s \cos \theta_{i,j}}} \quad (7.1)$$

با مقایسه کردن این دو تابع هزینه متوجه میشویم که ضریب منفی در L_{IAM} حذف شده است چون بجای بیشینه کردن کسینوس زاویه بردار وزن کلاس واقعی نمونه و خود نمونه، به دنبال کمینه کردن میانگین کسینوس زاویه ای برای بردار وزن کلاس غیر خودی و خود نمونه میباشد و یعنی بیشینه کردن فاصله زاویه ای کلاس غیر خودی و نمونه، این همون بحثی هست که بالاتر بیان کردیم و یعنی به دنبال بیشینه کردن فاصله بین کلاسی میباشد. در شکل ۲ این مقاله به وضوح میتوان نتایج و مقایسه این دو تابع هزینه را دید و بیشتر بودن فاصله بین کلاسی را دید.

دلیل اهمیت زیاد کردن واریانس برون کلاسی برای مسائل تشخیص چهره:

- افزایش این واریانس باعث می شود که مدل بیشتر بتواند تمایز پیدا کند بین آدم های متفاوت، حتی برای افرادی که شباهت ظاهری زیادی دارند، تفاوت های ساختاری پیدا کند.
- همینطور باعث میشود که کلاسترهای افراد از هم دور باشند، اگر این واریانس کم باشد و کلاسترها در هم بروند و باعث میشود و ویژگی های منحصر به فرد استخراج نکرده و مدل در زمان inference طبقه بندی اشتباه بیشتری داشته باشد.
- همینطور باعث میشود که در Hypersphere بین دو کلاس فضای خالی بیشتری باشد که وقتی چهره جدیدی دیده شد به اشتباه به یک کلاس موجود طبقه بندی نشود.

دلیل اهمیت بیشتر فاصله زاویه‌ای از فاصله اقلیدسی ساده:

- در فاصله اقلیدسی طول بردارها اهمیت دارد. طول بردار ویژگی $(||x||)$ تحت تأثیر کیفیت و یا روشنایی و یا نویز تصویر است که در مسئله طبقه بندی چهره اهمیتی ندارند. با استفاده از فاصله زاویه‌ای و نرمال‌سازی، مدل اثرات مخرب محیطی را حذف میکند.
- در فضای اقلیدسی بردارها محدودیتی بر اندازه ندارند و هر چقدر بخواهند زیاد میتوانند بشوند اما در فاصله زاویه‌ای ما با محدود کردن ویژگی‌ها به سطح یک کره و استفاده از ضریب s ، فرآیند بهینه‌سازی بسیار پایدارتر و سریع‌تر انجام می‌شود.
- در فضای اقلیدسی نمیتوان کلاس‌ها را از هم دور کرد، در روش‌هایی مانند L_2 -Constrained Softmax اندازه بردارها را محدود میکنند تا موارد قبلی مشکل ایجاد نکند اما باز به یک Hyper-sphere با شعاع ثابت محدود میشود و دیگر کنترلی بر این ندارد که کلاس‌های متفاوت را از هم دور نگه دارد. در فضای اقلیدسی Margin ای که تعریف میکنیم باعث میشود مدل بیشتر تلاش کند تا ویژگی‌های عمیق‌تر هر فرد را درک کند و به طور غیرمستقیم در فاصله بین کلاسی تأثیر می‌گذارد اما در فضای زاویه‌ای Margin به طور مستقیم مدل را مجبور می‌کند که کلاس‌ها را با یک فاصله زاویه‌ای مشخص از هم جدا کند.

۳.۱.۱ ارتباط فاصله زاویه‌ای به تشخیص چهره

میدانیم:

$$w^T x = ||w|| \cdot ||x|| \cdot \cos(\theta) \quad (۸.۱)$$

که در آن θ زاویه بین این دو بردار است. حال با نرمال سازی L_2 طول بردار ویژگی و وزن برابر با یک میشود $(||w|| = 1 \text{ و } ||x|| = 1)$ حال با کمک جایگذاری داریم:

$$\cos(\theta) = w^T x \quad (۹.۱)$$

این به این معناست که امتیاز مدل برای هر نمونه (Logits) را میتوان با فقط داشتن زاویه حساب کرد و عملاً مدل براساس شباهت کسینوسی دو بردار ویژگی و وزن کار میکند. کلاً اگر این دو بردار را نرمال کنیم، فاصله اقلیدسی بین این دو بردار تنها تابعی از زاویه میشود و نشان میدهد در یک ابرکره واحد فاصله اقلیدسی تنها

تابعی از θ می باشد.

$$\|w - x\|^2 = \|w\|^2 + \|x\|^2 - 2w^T x \quad (10.1)$$

$$\|w - x\|^2 = 1 + 1 - 2 \cos(\theta) = 2(1 - \cos(\theta)) \quad (11.1)$$

هرچه زاویه θ کمتر باشد، $\cos(\theta)$ به یک نزدیک تر شده و در نتیجه فاصله اقلیدسی کاهش می یابد. یعنی در ابرکره واحد، فاصله اقلیدوسی با زاویه رابطه یکنواخت دارد و بخاطر همین در فرمول تابع هزینه L_{IAM} دیگر با خیال راحت میتوانیم logit را با $\cos(\theta)$ نشان دهیم (ضریب ثابتی نیز ضرب می شود) و استفاده از فاصله زاویه ای در این فضای جدید (نرمال سازی شده بردار ها و ابرکره واحد) مشکلی ندارد.

پیش تر درباره اهمیت تغییر فضا برای تسک تشخیص چهره صحبت شده بود و میتوانید از زیربخش قبلی مطالعه بفرمایید ولی برای مرور:

- کاهش اثر عوامل محیطی: کیفیت و روشنایی و نویز هیچکدام در این تسک برای ما اهمیتی ندارد. طول بردار توسط این موارد تحت تاثیر قرار میگیرد پس مهم است که امتیاز مدل را مستقل از آن بکنیم (با ثابت کردن اندازه آن)، با این کار شبکه مجبور می شود تمام اطلاعات مربوط به هویت شخص را صرفاً در جهت بردار (زاویه) ذخیره کند (البته کامل مستقل نمیشود ولی خیلی از اثرش را میتوانیم حذف کنیم).
- درست طبقه بندی کردن فقط مهم نیست: برای robust بودن این تسک و درست عمل کردن مدل در صورت مواجهه با چهره جدید، ما نیاز داریم که در تشخیص چهره، ویژگی های چهره یک فرد به هم نزدیک باشند و از چهره افراد دیگر فاصله زیادی داشته باشند. وقتی فضای ویژگی را به یک فضای زاویه ای روی کره محدود می کنیم، به راحتی با وارد کردن مستقیم زاویه میتوان Margin های زاویه ای درست کرد و به صورت مستقیم باعث میشود که چهره آدم های متفاوت از هم دور بشوند (در فضای اقلیدسی ما باید فاصله را تعریف میکردیم و بعد عبارت های Regularization اضافه میکردیم که اندازه بردار ها همینطوری زیاد نشود که مدل گول بخورد چون میدانیم اندازه نشان دهنده هویت نیست طبق مورد قبلی و Margin اقلیدسی نیز می گذاشتیم که باز چون از جنس اندازه بود و نه زاویه، به طور غیرمستقیم چهره آدم های متفاوت رو از هم دور میکرد و برای جفت های سخت (آدم های مشابه ظاهری) کارش بسیار سخت تر میبود که دور کند آنها را از هم).

۴.۱.۱ تابع زیان IAM

در این مقاله، تابع زیان IAM به شکل زیر تعریف می‌شود:

$$L_{IAM} = \frac{1}{N} \sum_{i=1}^N \log \frac{\frac{1}{C-1} \sum_{j=1, j \neq y_i}^C e^{s \cos \theta_{i,j}}}{\sum_{j=1}^C e^{s \cos \theta_{i,j}}} \quad (۱۲.۱)$$

این تابع در زیربخش‌های قبلی به طور کامل توضیح داده شده است. در زیربخش اول تابع Softmax را بررسی کردیم و بعد تبدیل آن را به L_{scale} دیدیم (با نرمال‌سازی بردارها به فضای زاویه ای رفتیم) و بعد از آن در زیربخش دوم تبدیل L_{scale} به تابع L_{IAM} را دیدیم، برای توضیحات کامل این تابع لطفاً این دو زیربخش رو مطالعه بفرمایید، در اینجا صرفاً یک مرور مختصر آورده شده است:

- به دنبال محاسبه $P(\text{Data})$ بودیم و دیدیم بخاطر پیاده سازی راحت و مشکلات عددی، لگاریتم این عبارت را محاسبه کردن راحت‌تر می‌باشد و حاصل ضرب هم به حاصل جمع تبدیل می‌شود بعد از اعمال لگاریتم.
- حال داخل لگاریتم یک عبارتی داریم که می‌خواهیم کمینه کنیم. توجه کنید که در Softmax یا L_{scale} یک ضریب منفی پشت سیگما داشتند و یعنی به دنبال بیشینه کردن عبارت داخلشون بودند. در اینجا برعکس می‌باشد چون نحوه دید به مسئله نیز برعکس شده است.
- در L_{scale} صورت کسر کسینوس بردار ویژگی و وزن کلاس صحیح را سعی میکرد بیشینه کند و یعنی زاویه این دو را کمینه کند، اما این مشکل داشت چون فاصله برون کلاسی را زیاد نمیکرد بخاطر همین در L_{IAM} دیدگاه برعکس شد و در صورت کسر عبارت $\frac{1}{C-1} \sum_{j \neq y_i} e^{s \cos \theta_{i,j}}$ نشان‌دهنده میانگین شباهت کسینوسی ویژگی استخراج شده (x_i) با بردارهای وزن کلاس‌های غیر خودی (کلاس‌هایی غیر از کلاس حقیقی y_i) است و به دنبال کمینه کردن آن است.
- مخرج کسر همان مخرج استاندارد تابع Softmax است که مجموع نمایی شباهت با تمام کلاس‌ها را نشان می‌دهد.
- ضریب s ، در زیربخش‌های قبلی مفصل درباره این توضیح داده شده است ولی به طور کلی باعث آموزش بهتر و confidence بیشتر مدل می‌شود (ابعاد خروجی شبکه را بزرگتر میکند توسط یک ضریب ثابت تا هم‌گردیان راحت‌تر باشد و هم با قطعیت بیشتری تصمیمش را درباره کلاس درست بگوید.)

اگر زاویه بین نمونه x_i و یک کلاس نادرست w_j کوچک باشد (یعنی نمونه به کلاس اشتباه نزدیک باشد)، مقدار کسینوس بزرگ شده و در نتیجه صورت کسر مقدار بزرگ‌تری می‌گیرد. از آنجا که از این عبارت لگاریتم گرفته می‌شود، مقدار کل تابع زیان L_{IAM} افزایش می‌یابد. این تابع به تنهایی استفاده نمی‌شود، بلکه

به عنوان یک عبارت Regularization با ضریب β به توابع زیان پایه (مثل Softmax یا نسخه‌های زاویه‌ای آن) اضافه می‌شود:

$$L = L_{base} + \beta L_{IAM} \quad (۱۳.۱)$$

$$L = -\frac{1}{N} \sum_{i=1}^N \log(p_i) + \beta \frac{1}{N} \sum_{i=1}^N \log\left(\frac{1-p_i}{C-1}\right) \quad (۱۴.۱)$$

- عبارت N : اندازه دسته (Batch size).
- عبارت C : تعداد کل کلاس‌ها.
- عبارت $\bar{x}_i$: بردار ویژگی نرمال شده برای نمونه i .
- عبارت $\bar{w}_{y_i}$: بردار وزن نرمال شده برای کلاس صحیح (مربوط به نمونه i).
- عبارت $\bar{w}_j$: بردار وزن نرمال شده برای کلاس‌های نادرست ($j \neq y_i$).
- عبارت s : Scale factor.
- عبارت β : هایپرپارامتر عبارت Regularization.
- عبارت p_i : احتمال پیش‌بینی کلاس صحیح توسط Softmax. هرچه زاویه $\bar{x}_i$ با $\bar{w}_{y_i}$ کمتر باشد، این مقدار به ۱ نزدیک‌تر می‌شود:

$$p_i = \frac{e^{s\bar{w}_{y_i}^T \bar{x}_i}}{\sum_{k=1}^C e^{s\bar{w}_k^T \bar{x}_i}} \quad (۱۵.۱)$$

- عبارت p_{ij1} : احتمال پیش‌بینی کلاس j توسط Softmax (با در نظر گرفتن تمام کلاس‌ها در مخرج).

$$p_{ij1} = \frac{e^{s\bar{w}_j^T \bar{x}_i}}{\sum_{k=1}^C e^{s\bar{w}_k^T \bar{x}_i}} \quad (۱۶.۱)$$

- عبارت p_{ij2} : احتمال پیش‌بینی کلاس j در حالتی که کلاس صحیح را از مخرج حذف کنیم. این متغیر نشان‌دهنده سهم کلاس غلط j در میان تمام کلاس‌های غلط است.

$$p_{ij2} = \frac{e^{s\bar{w}_j^T \bar{x}_i}}{\sum_{k \neq y_i}^C e^{s\bar{w}_k^T \bar{x}_i}} \quad (۱۷.۱)$$

حال بیاید ببینیم اگر دو کلاس زاویه کمی با هم داشته باشند، loss IAM چه تغییری در گرادیان ایجاد می‌کند. یعنی که بردار ویژگی یک نمونه ($\bar{x}_i$) که متعلق به کلاس واقعی y_i است، در فضای ویژگی‌ها بیش از حد به بردار وزن یک کلاس اشتباه ($\bar{w}_j$) نزدیک شده باشد. با توجه به فرمول‌هایی که بررسی کردیم، تابع IAM این تغییر دقیق را در گرادیان ایجاد می‌کند:

اثرگذاری روی متغیر p_{ij2} ، وقتی زاویه بین ویژگی $\bar{x}_i$ و وزن کلاس اشتباه $\bar{w}_j$ کوچک باشد، مقدار ضرب داخلی آن‌ها ($\bar{w}_j^T \bar{x}_i$) بسیار بزرگ می‌شود. در نتیجه، مقدار نمایی در صورت کسر به شدت رشد می‌کند و p_{ij2} (که سهم این کلاس غلط را در بین تمام کلاس‌های غلط نشان می‌دهد) مقدار بسیار بزرگی به خود می‌گیرد. گرادیان مربوط به آپدیت وزن کلاس اشتباه در مقاله به این صورت آورده شده است:

$$\frac{\partial L}{\partial \bar{w}_j} = \frac{1}{N} \sum_{i=1}^N s \bar{x}_i (p_{ij1} + \beta p_{ij2} - \beta p_{ij1}) \quad (۱۸.۱)$$

در حالت Softmax معمولی، گرادیان فقط با p_{ij1} متناسب است. اما با اضافه شدن تابع IAM، جمله βp_{ij2} نیز هم اضافه شده است و از آنجایی که در صورت کوچک بودن زاویه بین دو کلاس، مقدار p_{ij2} به شدت بزرگ می‌شود، حضور عبارت βp_{ij2} باعث می‌شود قدر مطلق گرادیان (یعنی اندازه تغییرات وزن) افزایش یابد. یعنی اگر یک کلاس اشتباه زاویه کمی با نمونه دارد، گرادیان بسیار بزرگ‌تری برای آن تولید می‌کند تا در آپدیت بعدی وزن‌ها، آن کلاس اشتباه را با شدت بیشتری از نمونه دور کند و margin Inter-class را افزایش دهد.

آموزش مدل

- در شروع هر تکرار آموزش، یک Mini-batch با اندازه N به صورت تصادفی از مجموعه داده‌های آموزشی انتخاب می‌شود.
- این داده‌ها وارد شبکه استخراج‌کننده ویژگی (مانند شبکه Inception-ResNet-V۱ در مقاله) می‌شوند تا بردار ویژگی‌های آن‌ها (که با نماد x_i نشان داده می‌شود) تولید شود.
- سپس، هم بردارهای ویژگی (x_i) و هم بردارهای وزن مربوط به لایه دسته‌بند نهایی (w) با استفاده از نرمال‌سازی L۲ نرمال می‌شوند.
- میزان خطای مدل که ترکیبی از دو بخش، تابع هزینه پایه و تابع L_{IAM} Regularization (با یک ضریب β) محاسبه می‌شود.

$$L = L_{base} + \beta L_{IAM} \quad (۱۹.۱)$$

- خطا از انتهای شبکه به سمت عقب پخش می‌شود و گرادیان‌ها (مشتقات تابع هزینه) محاسبه می‌گردند. این گرادیان‌ها به صورت مجزا برای تابع پایه و تابع IAM محاسبه شده و سپس با هم جمع می‌شوند.
- با استفاده از یک الگوریتم بهینه‌ساز پارامترها برخلاف جهت گرادیان تغییر می‌کنند تا در دور بعدی خطای کمتری داشته باشند.
- به‌روزرسانی وزن‌های شبکه (ϕ) :

$$\phi \leftarrow \phi - r \frac{1}{N} \sum_{i=1}^N \left[\frac{\partial L_{base}}{\partial \bar{x}_i} + \beta \frac{\partial L_{IAM}}{\partial \bar{x}_i} \right] \frac{\partial x_i}{\partial \phi} \quad (20.1)$$

- به‌روزرسانی بردار وزن کلاس هدف (کلاس صحیح $\bar{w}_{y_i}$):

$$\bar{w}_{y_i} \leftarrow \bar{w}_{y_i} - r \frac{1}{N} \sum_{i=1}^N \left[\frac{\partial L_{base}}{\partial \bar{w}_{y_i}} + \beta \frac{\partial L_{IAM}}{\partial \bar{w}_{y_i}} \right] \quad (21.1)$$

- به‌روزرسانی بردار وزن کلاس‌های نادرست $(\bar{w}_j)$:

$$\bar{w}_j \leftarrow \bar{w}_j - r \frac{1}{N} \sum_{i=1}^N \left[\frac{\partial L_{base}}{\partial \bar{w}_j} + \beta \frac{\partial L_{IAM}}{\partial \bar{w}_j} \right] \quad (22.1)$$

این چرخه مجدداً برای دسته‌های جدیدی از داده‌ها تکرار می‌شود و تا زمانی که تعداد دفعات آموزش به حداکثر مقدار تعیین‌شده برسد، ادامه می‌یابد. الگوریتم دقیق ارائه شده در مقاله به شرح زیر است:

Algorithm 1 Learning with the IAM loss

Input: Training data $\{x_i\}$; training labels $\{y_i\}$; parameters ϕ of Inception-ResNet-V1; weights of classifier w ; learning rate r ; hyper-parameter β ; and maximum number of iterations t_m

Output: Parameters ϕ of Inception-ResNet-V1

```

1:  $t \leftarrow 1$ 
2: repeat
3:   Randomly select a mini-batch of size  $N$  from the training set
4:   Normalize the features and weights
5:   Compute the total loss  $L = L_{base} + \beta L_{IAM}$ 
6:   Update  $\phi$  by  $\phi \leftarrow \phi - r \frac{1}{N} \sum_{i=1}^N [\frac{\partial L_{base}}{\partial \bar{x}_i} + \beta \frac{\partial L_{IAM}}{\partial \bar{x}_i}] \frac{\partial x_i}{\partial \phi}$ 
7:   Update  $\bar{w}_{y_i}$  by  $\bar{w}_{y_i} \leftarrow \bar{w}_{y_i} - r \frac{1}{N} \sum_{i=1}^N [\frac{\partial L_{base}}{\partial \bar{w}_{y_i}} + \beta \frac{\partial L_{IAM}}{\partial \bar{w}_{y_i}}]$ 
8:   Update  $\bar{w}_j$  by  $\bar{w}_j \leftarrow \bar{w}_j - r \frac{1}{N} \sum_{i=1}^N [\frac{\partial L_{base}}{\partial \bar{w}_j} + \beta \frac{\partial L_{IAM}}{\partial \bar{w}_j}]$ 
9:    $t \leftarrow t + 1$ 
10: until  $t > t_m$ 

```

تعیین هایپرپارامتر

مهم‌ترین هایپرپارامتر در این روش، ضریب β Regularization است برای تعیین محدوده مناسب این پارامتر، مقاله رفتار گرادیان کلاس هدف (کلاس صحیح y_i) را بررسی می‌کند:

$$\frac{\partial L}{\partial \bar{w}_{y_i}} = -\frac{1}{N} \sum_{i=1}^N s \bar{x}_i (1 - p_i + \beta p_i) \quad (23.1)$$

تأثیر مقادیر مختلف β به شرح زیر است:

- **حالت $\beta = 0$:** تابع زیان IAM کاملاً غیرفعال می‌شود و مدل فقط با تابع زیان پایه (مثلاً Softmax) آموزش می‌بیند.
- **حالت $\beta = 1$:** عبارت داخل پرانتز به ۱ تبدیل شده و گرادیان برابر با $-s \bar{x}_i$ می‌شود. در این حالت بردار وزن کلاس ($\bar{w}_{y_i}$) صرفاً به سمت نمونه‌ها حرکت می‌کند.

• **حالت $\beta < 1$:** اضافه شدن عبارت βp_i باعث می‌شود قدر مطلق گرادیان نسبت به حالت پایه ($\beta = 0$) مقداری افزایش یابد. در این حالت باعث کاهش فاصله درون‌کلاسی و افزایش واریانس بین‌کلاسی می‌شود.

• **حالت $\beta > 1$:** هرچه p_i بزرگتر شود (یعنی نمونه از قبل زاویه کمی با کلاس خود داشته باشد و خوب طبقه‌بندی شده باشد)، قدر مطلق گرادیان بیشتر می‌شود (طبق معادل فوق) این رفتار کاملاً نامطلوب است زیرا تغییرات وزن‌ها بیشتر می‌شود با کاهش فاصله درون‌کلاسی و این رفتار متناقض هست. برعکس این نیز دیده می‌شود یعنی تغییرات وزن‌ها نیز کمتر می‌شود با افزایش فاصله برون‌کلاسی (یعنی p_{ij2} کاهش می‌یابد و β نیز بیشتر کاهشش می‌دهد و تغییرات وزن کمتر می‌شود طبق معادله زیر):

$$\frac{\partial L}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N s\bar{x}_i (p_{ij1} + \beta p_{ij2} - \beta p_{ij1}) \quad (24.1)$$

بنابراین پارامتر β حتماً باید مقداری کوچک‌تر از ۱ داشته باشد تا تابع IAM بتواند عملکرد توابع زیان پایه را بهبود بخشد.

۵.۱.۱ دیگر توابع زیان

تابع زیان Sphere

این تابع زیان، بردار وزن‌ها را نرمال‌سازی می‌کند و یک Margin زاویه‌ای از طریق ضرب کردن یک ضریب در زاویه ایجاد می‌کند.

$$L_{sphere} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{\|x_i\| \cos(m\theta_{i,y_i})}}{e^{\|x_i\| \cos(m\theta_{i,y_i})} + \sum_{j=1, j \neq y_i}^C e^{\|x_i\| \cos \theta_{i,j}}} \quad (25.1)$$

• **نرمال‌سازی بردار وزن‌ها:** بسیار مهم است و اگر نبود، شبکه متوجه می‌شود که برای کاهش خطای یک کلاس (مثلاً کلاسی که نمونه‌های آموزشی بیشتری دارد یا یادگیری آن آسان‌تر است)، می‌تواند به جای کاهش زاویه، صرفاً طول بردار وزن آن کلاس را بزرگتر بکند، در این صورت bias بین کلاس‌ها ایجاد می‌شود و باعث طبقه‌بندی اشتباه می‌شود.

• **عدم نرمال‌سازی ویژگی‌ها در ابتدا:** در ابتدا وقتی این تابع معرفی شد، متعقد بودند که بردار ویژگی رو نباید نرمال‌سازی کنیم چون اطلاعات مهمی درونش هست اما مشاهده شد آموزش بسیار ناپایدار هست چون مدل بجای اینکه تلاش کند کسینوس را بیشینه کند (یعنی زاویه رو کمینه کند)، راه راحت‌تر

براش این هست که اندازه بردار ویژگی رو زیاده‌تر کند و همینطور دیده شد بردار ویژگی ارتباط مستقیم با کیفیت و روشنایی تصویر دارد پس کمکی نمیکرد که آموزش این سمتی برود. توجه کنید که گردیان های نمونه های آسون (کیفیت خوب و روشن) چون اندازه بردار ویژگی بیشتری دارند، بیشتر بود و در مقابل نمونه های سخت خیلی بیشتر بود و عملاً در آپدیت شدن وزن ها فقط توجه به این نمونه های آسون بکند.

- **نحوه عملکرد Margin:** در مخرج کسر برای کلاس های نادرست $(y_i \neq j)$ ، از همان $\cos(\theta_{i,j})$ استفاده شده است، اما برای کلاس صحیح (y_i) ، زاویه در یک هاپرپارامتر m (که بزرگتر از ۱ است) ضرب شده است $(m\theta_{i,y_i})$. شبکه میخواهد کسینوس زاویه را زیاد بکند و یعنی عبارت $m\theta_{i,y_i}$ را کاهش دهد و چون m بزرگتر از یک است، مجبور است نسبت به حالت عادی θ_{i,y_i} را بیشتر کاهش دهد تا زیان را به حداقل برساند.
- **کاهش فاصله درون کلاسی:** وقتی شبکه می بیند زاویه θ در عدد m ضرب شده است، احساس می کند که نمونه از مرکز کلاس خود بسیار دور است (حتی اگر واقعاً این طور نباشد). برای جبران این، شبکه بردار ویژگی را به سمت مرکز کلاس می کشد. این کار باعث می شود نمونه های یک کلاس به شدت فشرده شده و فاصله درون کلاسی آن ها بسیار کوچک شود.
- **افزایش فاصله بین کلاسی:** از آنجایی که فضای روی ابرکره محدود است، وقتی هر کلاس در یک ناحیه بسیار تنگ و فشرده جمع شود، به طور طبیعی فضای خالی بین کلاس ها بیشتر شده و مرزهای بین کلاسی بزرگتر می شوند.
- **دشواری بهینه سازی:** از آنجا که margin به صورت ضربی $(m\theta)$ است، میزان جریمه به خود زاویه بستگی دارد. این رفتار غیرخطی و پیچیده باعث می شود فضای بهینه سازی بسیار دشوار شود (بخاطر همین به عنوان تابع هزینه تنها استفاده نمیشود و به عنوان عبارت Regularization بکار می رود).
- **مرز تصمیم گیری:** برای حالت طبقه بندی باینری، مرز این تابع برابر با $\theta_{i,1} = m\theta_{i,1}$ است (با فرض اینکه کلاس ۱ کلاس صحیح باشد). این یعنی فاصله زاویه ای تا کلاس صحیح باید m برابر کمتر از فاصله تا کلاس اشتباه باشد.

تابع زیان Cos

این تابع زیان به جای ضرب کردن زاویه در یک ضریب، Margin را مستقیماً از مقدار کسینوس زاویه کلاس صحیح کم می کند.

$$L_{cos} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s(\cos \theta_{i,y_i} - m)}}{e^{s(\cos \theta_{i,y_i} - m)} + \sum_{j=1, j \neq y_i}^C e^{s \cos \theta_{i,j}}} \quad (26.1)$$

- در اینجا هم وزن‌ها و هم ویژگی‌ها نرمال شده‌اند (همانطور که بالاتر گفتیم چرا این کار بهتر است) و کل عبارت در یک فاکتور مقیاس s ضرب شده است (دلیل آن در زیربخش اول اومده است).
- با کم کردن مقدار ثابت m از $\cos(\theta_{i,y_i})$ ، ما مقدار صورت کسر را کاهش می‌دهیم. برای اینکه شبکه بتواند زیان را کم کند میبایست عبارت درونی را بیشینه کند و حال چون ما دستی m تا کاهش دادیم، شبکه باید $\cos(\theta_{i,y_i})$ را به سمت ۱ (یعنی زاویه صفر) سوق دهد.
- برخلاف تابع زیان Sphere که Margin متغیر بود، در اینجا یک یک فاصله کاملاً ثابت و یکنواخت به اندازه m در فضای کسینوسی بین کلاس صحیح و سایر کلاس‌ها ایجاد می‌کند که باعث بهینه‌سازی راحت‌تری میشود.
- برای اینکه شبکه بتواند تابع زیان را کاهش دهد، نمونه نه‌تنها باید به کلاس خودش نزدیک‌تر از بقیه کلاس‌ها باشد، بلکه باید حداقل به اندازه m از نزدیک‌ترین کلاس اشتباه فاصله داشته باشد. این اجبار همزمان باعث کاهش فاصله درون‌کلاسی و افزایش فاصله بین‌کلاسی می‌شود.
- مرز تصمیم‌گیری: در حالت دودویی، مرز برابر است با $\cos \theta_{i,1} - m = \cos \theta_{i,2}$ (با فرض اینکه کلاس ۱ کلاس صحیح باشد). این یعنی کسینوس زاویه با کلاس ۱ باید حداقل به اندازه مقدار m از کسینوس زاویه با کلاس ۲ بزرگتر باشد تا آن را در کلاس ۱ بپذیرد.

تابع زیان Arc

این تابع زیان بسیار شبیه به تابع زیان Cos است، اما به جای کسر کردن از مقدار کسینوس، m margin را مستقیماً به خود زاویه کلاس صحیح اضافه می‌کند.

$$L_{arc} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s \cos(\theta_{i,y_i} + m)}}{e^{s \cos(\theta_{i,y_i} + m)} + \sum_{j=1, j \neq y_i}^C e^{s \cos \theta_{i,j}}} \quad (27.1)$$

- با اضافه کردن m به θ_{i,y_i} ، زاویه بزرگتر شده و مقدار کسینوس آن کاهش می‌یابد. بنابراین مدل وادار می‌شود برای جبران، زاویه اصلی θ را به شدت کوچک کند.
- مرز تصمیم‌گیری: در این حالت، مرز تصمیم برای کلاس دودویی به شکل $\theta_{i,1} + m = \theta_{i,2}$ درمی‌آید.

اصلاحات:

- نسبت به تابع زیان Sphere: ضرب کردن زاویه باعث ایجاد یک مرز تصمیم وابسته به زاویه ($m\theta_1$) می‌شود. این کار رفتار گرادیان‌ها را بسیار تند، غیرخطی و ناپایدار می‌کند (به همین دلیل Sphere به سختی همگرا می‌شد و به کمک Softmax نیاز داشت). Arc با تبدیل این حاشیه به یک جمع ساده،

مرز تصمیم را به شکل خطی و پایدار $\theta_1 + m = \theta_2$ درمی‌آورد و فرآیند آموزش را بسیار نرم‌تر و پایدارتر می‌کند.

- نسبت به تابع زیان Cos به صورت هندسی میتوانیم دقیق‌تر باشیم. هدف اصلی تمام این توابع، جدا کردن کلاس‌ها روی یک Hypersphere است. واحد اندازه‌گیری فاصله در اینجا زاویه است و نه کسینوس آن. وقتی ما مقدار m را از کسینوس کم می‌کنیم، به دلیل غیرخطی بودن تابع کسینوس، حاشیه‌ای که روی کره ایجاد می‌شود یکنواخت نیست (مقادیر مختلفی از زاویه می‌توانند یک مقدار یکسان از کسینوس درست بکنند).

تفاوت تابع زیان IAM با SphereFace و NormFace

- تابع NormFace (همان L_{scale}) تنها پیشنهاد می‌کند که ویژگی‌ها و وزن‌ها نرمال شوند و یک ضریب s به آن‌ها اضافه شود تا شبکه راحت‌تر همگرا شود یعنی صرفاً شباهت کسینوسی را بهینه می‌کند و اصلاً مکانیزمی برای افزایش margin بین‌کلاسی (Inter-class margin) در خود ندارد. اما IAM مستقیماً برای افزایش این margin طراحی شده است و می‌تواند عملکرد NormFace را به شدت ارتقا دهد.
- تابع SphereFace یک margin زاویه‌ای وارد می‌کند و چون ضرب میشود در زاویه، تاثیر آن متغیر بوده و بهینه‌سازی را سخت میکند ولی margin ای که در IAM بجای اضافه کردن یا ضرب کردن یک ضریب، به صورت Adaptive آورده شده است (جلوتر درباره این مورد بیشتر می‌پردازیم). همچنین در SphereFace بردار ویژگی را نرمال‌سازی نکردیم ولی در IAM شده است.
- جفت روش‌های بالا به دنبال بیشینه کردن کسینوس زاویه بردار ویژگی با بردار وزن کلاس صحیح هستند (زاویه کمتر) ولی در IAM دیدگاه برعکس هست یعنی به دنبال کاهش کسینوس زاویه بردار ویژگی با بردار وزن کلاس‌های اشتباه میباشد (زاویه بیشتر).

هدف تابع زیان IAM

تمام روش‌های قبلی، برای جدا کردن کلاس‌ها از هم یک margin ثابت مانند m معرفی می‌کنند. آن‌ها به اینکه زاویه فعلی بین کلاس‌های مختلف چقدر است، توجهی ندارند و یک جریمه ثابت را برای همه اعمال می‌کنند.

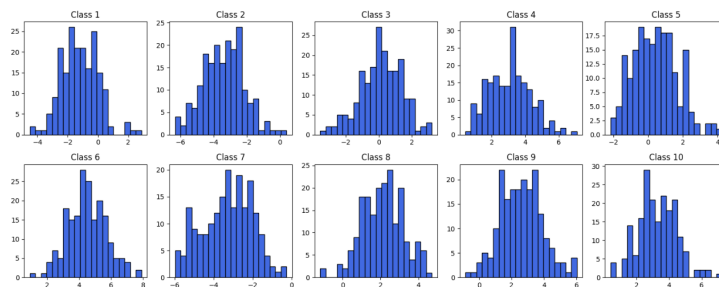
هدف اصلی IAM این هست که margin بین‌کلاسی را به صورت Adaptive افزایش دهد. این یعنی شبکه با استفاده از IAM متوجه می‌شود کدام کلاس‌ها به هم نزدیک‌تر هستند (زاویه کمتری دارند) و آن‌ها را با شدت بسیار بیشتری جریمه کرده و از هم دور می‌کند. هدف دیگر آن‌ها این بود که یک تابع درست نکنند بلکه یک عبارت Regularization انعطاف‌پذیر که بتواند به تمامی توابع زیان قبلی اضافه شده و دقت آن‌ها را در

تشخیص افزایش دهد.

۲.۱ پیاده سازی

۱.۲.۱ آماده سازی داده

در این بخش، یک مجموعه داده مصنوعی برای مسئله طبقه بندی تصاویر ایجاد شد. هدف، تولید داده هایی برای ۱۰ کلاس مجزا بود که هر کلاس نشان دهنده یک شخص متفاوت است. برای هر کلاس، ۲۰۰۰ نمونه تصویر با ابعاد ۱۴ در ۱۴ پیکسل تولید شد. روند تولید به این صورت بود که ابتدا برای هر کلاس یک میانگین تصادفی از توزیع یکنواخت در بازه $[-5.0, 5.0]$ استخراج گردید. سپس، پیکسل های تمامی ۲۰۰۰ نمونه مربوط به آن کلاس، از یک توزیع نرمال با همان میانگین تولید شده و واریانس ثابت 1.5 استخراج شدند. پس از تولید مجموعاً ۲۰۰۰۰ نمونه داده، برچسب کلاس ها با اعدادی از ۱ تا ۱۰ مشخص گردید. به منظور ارزیابی مدل ها، داده ها با نسبت ۸۰ درصد برای آموزش (۱۶۰۰۰ نمونه) و ۲۰ درصد برای تست (۴۰۰۰ نمونه) تقسیم شدند. برای حفظ تعادل تعداد نمونه های هر کلاس در هر دو مجموعه، از روش تقسیم یکنواخت (stratified) استفاده شد. ابعاد تنسورهای استخراج شده به شرح زیر است: ابعاد داده های آموزش: (16000, 14, 14) ابعاد داده های تست: (4000, 14, 14) ابعاد برچسب های آموزش: (16000,) ابعاد برچسب های تست: (4000,) ابعاد میانگین کلاس ها در مجموعه آموزش: (10, 196) توزیع داده ها در هر کلاس هیستوگرام زیر، توزیع مقادیر پیکسل ها را برای یک نمونه تصادفی از هر یک از ۱۰ کلاس نشان می دهد.



شکل ۱.۱: توزیع مقادیر پیکسل ها برای یک نمونه تصادفی از هر یک از ۱۰ کلاس

همان طور که در نمودارها مشخص است، داده های هر نمونه به خوبی از یک توزیع نرمال پیروی می کنند. مرکز این توزیع ها برای کلاس های مختلف متفاوت است که نشان دهنده همان میانگین تصادفی تخصیص یافته به هر کلاس است. برای بررسی میزان استقلال و همبستگی کلاس ها با یکدیگر، ماتریس کوواریانس میانگین کلاس ها محاسبه و رسم شد. در این تحلیل، منظور از میانگین کلاس، میانگین گرفته شده روی تمام نمونه های

مربوط به یک کلاس خاص در مجموعه داده آموزش است. ابتدا بررسی می‌کنیم که مقادیر روی قطر اصلی ماتریس کوواریانس باید چه مقداری داشته باشند. می‌دانیم که کوواریانس یک متغیر با خودش، برابر با واریانس آن است. فرض کنید یکی از ویژگی‌ها (یکی از ۱۹۶ پیکسل) را با متغیر تصادفی X نشان دهیم. ما N نمونه (در اینجا ۲۰۰۰ نمونه) از این ویژگی داریم و واریانس هر کدام برابر با ۱.۵ است:

$$Var(X_i) = \sigma^2 = 1.5 \quad (28.1)$$

مقداری که ماتریس کوواریانس روی آن محاسبه می‌شود، بردار میانگین کلاس است که آن را با $\bar{X}$ نشان می‌دهیم:

$$\bar{X} = \frac{X_1 + X_2 + \dots + X_N}{N} = \frac{1}{N} \sum_{i=1}^N X_i \quad (29.1)$$

با محاسبه واریانس این میانگین داریم:

$$Var(\bar{X}) = Var\left(\frac{1}{N} \sum_{i=1}^N X_i\right) \quad (30.1)$$

$$Var(\bar{X}) = \frac{1}{N^2} Var\left(\sum_{i=1}^N X_i\right) \quad (31.1)$$

با توجه به اینکه نمونه‌های تولید شده از یکدیگر مستقل هستند، واریانس مجموع برابر با مجموع واریانس‌ها خواهد بود:

$$Var(\bar{X}) = \frac{1}{N^2} \sum_{i=1}^N Var(X_i) \quad (32.1)$$

$$Var(\bar{X}) = \frac{1}{N^2} (N\sigma^2) \quad (33.1)$$

$$Var(\bar{X}) = \frac{\sigma^2}{N} \quad (34.1)$$

با جایگذاری مقادیر $\sigma^2 = 1.5$ و $N = 2000$ نتیجه می‌شود:

$$Var(\bar{X}) = \frac{1.5}{2000} = 0.00075 \quad (۳۵.۱)$$

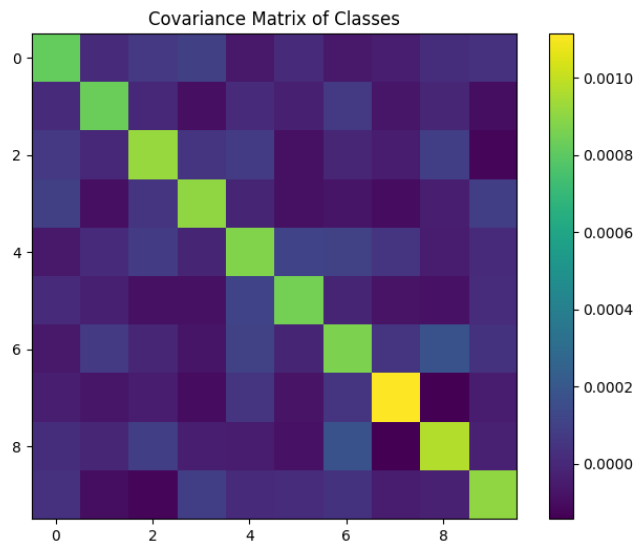
حال به محاسبه کوواریانس برای میانگین‌های دو کلاس متفاوت i و j می‌پردازیم. از آنجایی که در روند تولید داده‌ها، محاسبه و تولید نمونه‌های کلاس i بدون استفاده از هیچ اطلاعاتی از کلاس j انجام شده است، این دو متغیر تصادفی کاملاً مستقل از یکدیگر هستند. بنابراین برای دو متغیر تصادفی مستقل داریم:

$$E[\bar{X}_1 \bar{X}_2] = E[\bar{X}_1] \cdot E[\bar{X}_2] \quad (۳۶.۱)$$

در نتیجه مقدار کوواریانس برابر خواهد بود با:

$$Cov(\bar{X}_1, \bar{X}_2) = E[\bar{X}_1 \bar{X}_2] - (E[\bar{X}_1] \cdot E[\bar{X}_2]) = 0 \quad (۳۷.۱)$$

نتایج عملی ماتریس کوواریانس:

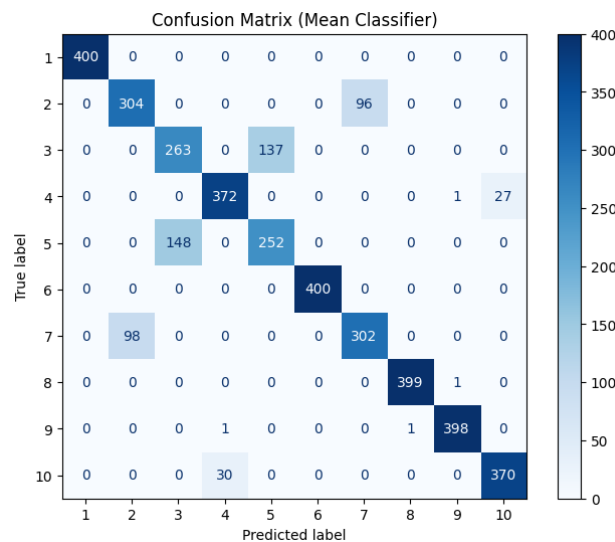


شکل ۲.۱: نتایج عملی ماتریس کوواریانس

همان‌طور که در تصویر ماتریس کوواریانس رسم شده مشاهده می‌شود، مقادیر روی قطر اصلی همگی در حدود 0.00075 هستند و مقادیر خارج از قطر اصلی همگی مقادیری بسیار نزدیک به صفر دارند. این امر به طور کامل اثبات تئوری بالا را تایید کرده و نشان می‌دهد کلاس‌ها از نظر آماری کاملاً مستقل تولید شده‌اند و

data leakage وجود ندارد. طبقه‌بند میانگین برای ایجاد یک baseline اولیه پیش از ورود به شبکه‌های عصبی پیچیده، از یک طبقه‌بند مبتنی بر میانگین استفاده شد. روند کار این طبقه‌بند به این شکل است که ابتدا با استفاده از داده‌های آموزش، بردار میانگین هر یک از ۱۰ کلاس محاسبه می‌شود (که ابعاد ۱۰ در ۱۹۶ دارد). برای هر نمونه از داده‌های تست، فاصله اقلیدسی آن تا میانگین تک تک کلاس‌ها محاسبه شده و کلاسی که کمترین فاصله را داشته باشد به عنوان پیش‌بینی نهایی برگردانده می‌شود. دقت به دست آمده روی داده‌های تست برای این طبقه‌بند برابر با 86.5 درصد است.

بررسی Confusion Matrix:



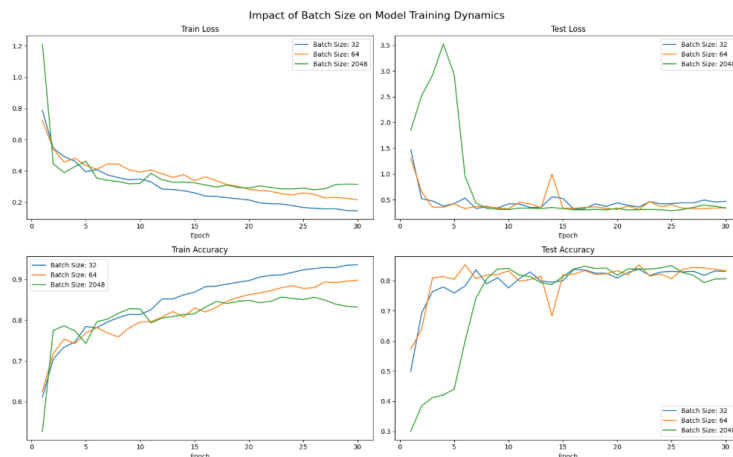
شکل ۳.۱: Confusion Matrix طبقه‌بند میانگین

با بررسی Confusion Matrix می‌توان به نقاط قوت و ضعف این طبقه‌بند پی برد. مقادیر روی قطر اصلی نشان‌دهنده تشخیص‌های درست هستند که در اکثر کلاس‌ها مقادیر بالایی را ثبت کرده‌اند (مثلاً کلاس ۱ و کلاس ۶ با دقت ۱۰۰ درصدی دسته‌بندی شده‌اند). اما نکته قابل توجه در مقادیر خارج از قطر اصلی است. خطاهای این طبقه‌بند به صورت تصادفی پخش نشده‌اند، بلکه بین جفت کلاس‌های خاصی تمرکز دارند. به عنوان مثال: کلاس ۳ و کلاس ۵ به شدت با یکدیگر اشتباه گرفته شده‌اند (کلاس ۵ به اشتباه ۱۴۸ بار در کلاس ۳ پیش‌بینی شده و کلاس ۳ نیز ۱۳۷ بار به اشتباه در کلاس ۵ قرار گرفته است). خطای مشابهی بین کلاس ۲ و کلاس ۷ نیز با نرخ بالایی (۹۶ و ۹۸ بار خطا) رخ داده است. دلیل این پدیده به نحوه تولید داده‌ها بازمی‌گردد. میانگین کلاس‌ها به صورت کاملاً تصادفی انتخاب شده‌اند. برای مثال میانگین کلاس ۳ و کلاس ۵ در فضای ۱۹۶ بعدی بسیار به یکدیگر نزدیک شده است. با توجه به واریانس نسبتاً بالای داده‌ها (1.5)،

توزیع فضایی نمونه‌های این دو کلاس دچار هم‌پوشانی شدیدی شده است. از آنجایی که طبقه‌بند میانگین صرفاً بر اساس کمترین فاصله اقلیدسی تصمیم‌گیری می‌کند، در مرزهای هم‌پوشانی دچار سردرگمی شده و نمونه‌های مرزی را به اشتباه در کلاس مجاور قرار می‌دهد.

۲.۲.۱ مدل پایه CNN با Softmax

در این بخش یک معماری شبکه CNN استاندارد به عنوان مدل Baseline طراحی و پیاده‌سازی شد. معماری این شبکه از شبکه MNIST مقاله الهام گرفته شده است (با تغییری کوچک به دلیل اینکه داده‌های MNIST ابعاد ۲۸ در ۲۸ دارند ولی در این پروژه ابعاد ۱۴ در ۱۴ است) و شامل سه بلوک اصلی استخراج ویژگی می‌باشد. در هر بلوک از دو لایه Conv2d، لایه‌های BatchNorm2d جهت تسریع همگرایی و پایداری آموزش، و توابع فعال‌ساز ReLU استفاده شده است. در انتهای هر بلوک نیز یک لایه MaxPool2d برای کاهش ابعاد فضایی قرار دارد. پس از استخراج ویژگی‌ها، تنسورها Flatten شده و به یک لایه تمام‌متصل با ۱۲۸ نورون وارد می‌شوند. خروجی این لایه همان فضای ویژگی (Embedding) ماست. در نهایت، لایه Classifier با ۱۰ نورون خروجی (معادل تعداد کلاس‌ها) قرار گرفته است. تعداد کل پارامترهای آموزش‌پذیر این شبکه ۳۵۴،۲۸۲ پارامتر است. مدل با استفاده از تابع زیان CrossEntropyLoss و بهینه‌ساز Adam به مدت ۳۰ اپیاک آموزش داده شد. برای کنترل نرخ یادگیری از یک Scheduler از نوع ReduceLROnPlateau استفاده شد تا در صورت توقف کاهش خطای تست، نرخ یادگیری را کاهش دهد. به منظور بررسی تاثیر اندازه دسته، فرآیند آموزش با سه مقدار متفاوت برای Batch Size شامل ۳۲، ۶۴ و ۲۰۴۸ تکرار گردید.

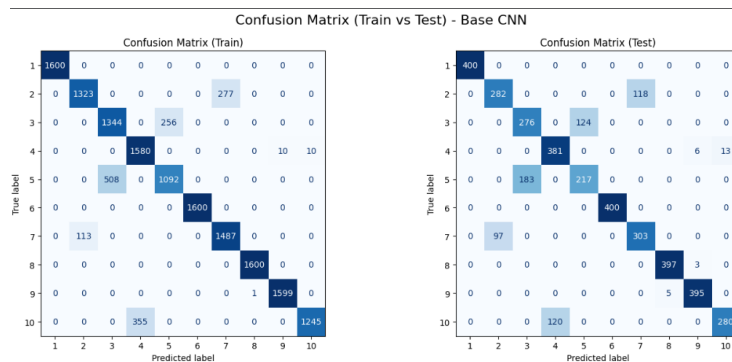


شکل ۴.۱: نمودارهای تغییرات تابع زیان و دقت برای مجموعه‌های آموزش و تست در طول ۳۰ اپیاک، به تفکیک اندازه‌های دسته متفاوت.

تحلیل تاثیر تغییر Batch Size: دسته Batch Size = ۳۲: مدل با این اندازه دسته بسیار سریع روی

داده‌های آموزشی همگرا می‌شود (دقت آموزش در پایان ایپاک ۳۰ به 93.61٪ رسیده است). اما با نگاه به نمودار خطای تست متوجه می‌شویم که از حدود ایپاک ۱۵ به بعد، خطای تست روند صعودی به خود گرفته است که نشان‌دهنده شروع پدیده Overfitting است. دسته $\text{Batch Size} = 64$: این حالت بهترین تعادل را بین سرعت همگرایی و پایداری نشان می‌دهد. شیب کاهش خطا نرم‌تر است و فاصله میان خطای آموزش و تست منطقی‌تر پیش رفته است. این حالت از دسته ۳۲ منطقی‌تر است زیرا فاصله میان خطای آموزش و تست در ایپاک آخر کمتر است، یعنی مدل روی داده‌های آموزش کمتر Overfit شده است. دسته $\text{Batch Size} = 2048$: به دلیل تعداد بسیار کم به‌روزرسانی‌های وزن در هر ایپاک، مدل در ایپاک‌های اولیه دچار نوسانات شدید شده و مسیر همگرایی آن بهینه و هموار نیست. در نتیجه این حالت از انتخاب‌های ما حذف می‌شود. مدل با $\text{Batch Size} = 64$ به عنوان مدل نهایی انتخاب گردید که به دقت نهایی آموزش 89.83٪ و دقت تست 83.28٪ رسید. متریک‌های کلان روی داده‌های تست برای این مدل شامل Macro Precision معادل 0.8383 و Macro Recall معادل 0.8328 بود. همچنین فاصله میانگین درون‌کلاسی 0.1840 و فاصله میانگین بین‌کلاسی 1.3557 به دست آمد.

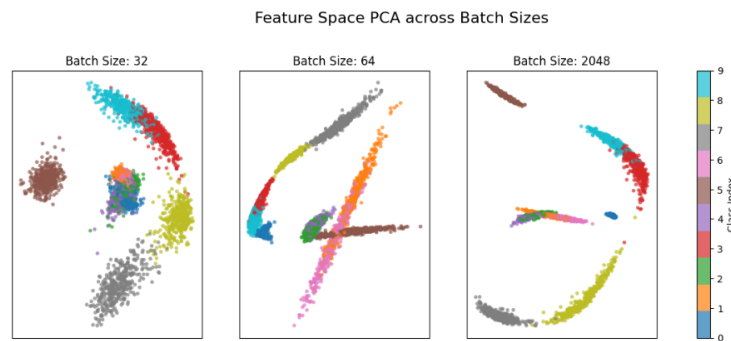
در محاسبه فواصل درون‌کلاسی و بین‌کلاسی، ویژگی‌ها ابتدا نرمال‌سازی شده‌اند. این نرمال‌سازی صرفاً در مرحله استخراج متریک‌ها انجام شده و در هیچ بخش دیگری از فرآیند آموزش یا رسم نمودارهای PCA از آن استفاده نشده است. هدف از این کار، فراهم کردن امکان مقایسه معنادار و عادلانه با مدل Angular Margin در بخش‌های بعدی است که ذاتاً در فضای نرمال‌سازی شده عمل می‌کند. برای بررسی دقیق‌تر عملکرد مدل و توزیع خطاها، Confusion Matrix برای هر دو مجموعه داده آموزش و تست (مربوط به مدل با بچ سائز ۶۴) رسم شد.



شکل ۵.۱: مقایسه Confusion Matrix مدل CNN روی داده‌های آموزش و تست.

در ماتریس آموزش، مدل توانسته بیشتر کلاس‌ها را با دقت بسیار بالا (اعداد روی قطر اصلی) یاد بگیرد. اما در ماتریس تست، برخی کلاس‌ها با افت شدید دقت مواجه شده‌اند. این تفاوت عملکرد روی داده‌های دیده

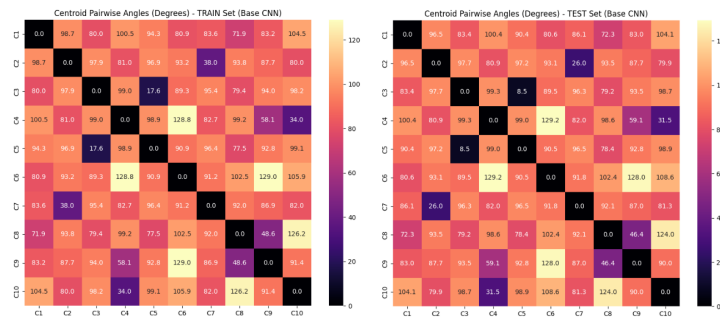
شده و دیده نشده، نشانگر Overfitting مدل روی ویژگی‌های خاص داده‌های آموزش است. آیا کلاس‌های نزدیک از نظر میانگین، بیشتر با هم اشتباه می‌شوند؟ بله، خطاهای مدل کاملاً هدفمند و متمرکز روی جفت کلاس‌های خاصی است. با بررسی ماتریس تست متوجه می‌شویم که شدیدترین خطاها بین کلاس‌هایی رخ داده است که میانگین‌های اولیه نزدیکی داشته‌اند (این نزدیکی تصادفی مراکز کلاس‌ها در فضای تولید داده اولیه رخ داده است مثلاً کلاس ۵ و ۳). شبکه CNN با تابع Softmax نتوانسته مرز تصمیمی با حاشیه کافی برای جدا کردن این کلاس‌های درهم‌تنیده بسازد. خروجی ۱۲۸ بعدی لایه قبل از طبقه‌بند استخراج شد و با استفاده از الگوریتم PCA به دو بعد کاهش یافت.



شکل ۶.۱: نگاشت دوبعدی فضای ویژگی با استفاده از الگوریتم PCA برای اندازه‌های دسته مختلف.

تحلیل جداسازی کلاس‌ها و میزان هم‌پوشانی: همان‌طور که در نمودارها (به ویژه برای دسته‌های ۳۲ و ۶۴) مشاهده می‌شود، توزیع داده‌ها در فضای ویژگی حالت شعاعی دارد. این رفتار مختص تابع Softmax است که برای افزایش اطمینان پیش‌بینی، طول بردارها، یعنی مقادیر $\|x\|$ ، را در فضا می‌کشد اما قیدی روی زاویه بین آن‌ها ندارد. جداسازی کلاس‌ها: کلاس‌هایی که میانگین متفاوتی داشته‌اند از مبدأ مختصات به سمت بیرون کشیده شده‌اند و تا حدودی قابل تفکیک هستند. میزان هم‌پوشانی: در مرکز مختصات هم‌پوشانی بالایی بین کلاس‌ها وجود دارد. مهم‌تر از آن، کلاس‌های نزدیک که در Confusion Matrix هم بیشترین خطا را داشتند، در فضای ویژگی روی یکدیگر یا بسیار نزدیک به هم قرار گرفته‌اند (مثل کلاس ۳ و ۵ که متناظر رنگ سبز و بنفش هستند). نبود یک حاشیه تضمین شده باعث می‌شود داده‌های تست مربوط به کلاس‌های نزدیک، به راحتی از مرز تصمیم عبور کرده و اشتباه دسته‌بندی شوند. تحلیل زوایای بین کلاس‌ها برای بررسی دقیق‌تر نحوه قرارگیری کلاس‌ها در فضای ویژگی، نقشه حرارتی زاویه بین مراکز کلاس‌ها محاسبه و رسم گردید.

در تابع هزینه Softmax معمولی، بهینه‌سازی شبکه صرفاً بر اساس بیشینه‌سازی ضرب داخلی بردار ویژگی و بردار وزن کلاس صحیح انجام می‌شود و شبکه هیچ اجبار یا جریمه مستقیمی برای دور کردن زوایای کلاس‌های متفاوت از یکدیگر دریافت نمی‌کند. به همین دلیل در نقشه حرارتی مشاهده می‌کنیم که زاویه بین



شکل ۷.۱: نقشه حرارتی زاویه به درجه بین مراکز کلاسها در فضای ویژگی برای داده‌های آموزش و تست.

برخی از کلاسها بسیار کوچک است و توزیع زوایا ساختار منظمی ندارد. این مقادیر دقیقاً متناظر با همان کلاس‌هایی است که در فضای PCA با هم هم‌پوشانی داشتند. این نقشه حرارتی نقطه مبنای مناسبی ایجاد می‌کند تا در بخش‌های بعدی اثر اعمال حاشیه زاویه‌ای بر باز شدن و افزایش فاصله بین کلاسها ارزیابی گردد.

۳.۲.۱ پیاده‌سازی Angular Margin Loss

در این بخش، معماری شبکه عصبی پیشین بدون تغییر باقی ماند و تنها تابع هزینه از Softmax معمولی به یک تابع هزینه مبتنی بر حاشیه زاویه‌ای (Angular Margin Loss) تغییر یافت تا تاثیر مستقیم اعمال حاشیه بر فضای ویژگی ارزیابی شود. ۱. نرمال سازی ویژگی و وزن‌ها در قدم اول، خروجی ۱۲۸ بعدی لایه استخراج ویژگی (Embedding) و همچنین وزن‌های لایه نهایی با استفاده از نرم L2 نرمال سازی شدند. دلیل این کار و رسیدن به فاصله زاویه‌ای به صورت ریاضی در زیر اثبات شده است: می‌دانیم که ضرب داخلی دو بردار x (ویژگی) و w (وزن) برابر است با:

$$w^T x = ||w|| ||x|| \cos(\theta) \quad (38.1)$$

از آنجا که هر دو بردار به صورت اجباری نرمال شده‌اند، اندازه آن‌ها برابر با ۱ است، یعنی:

$$||w|| = 1 \quad (39.1)$$

$$||x|| = 1 \quad (40.1)$$

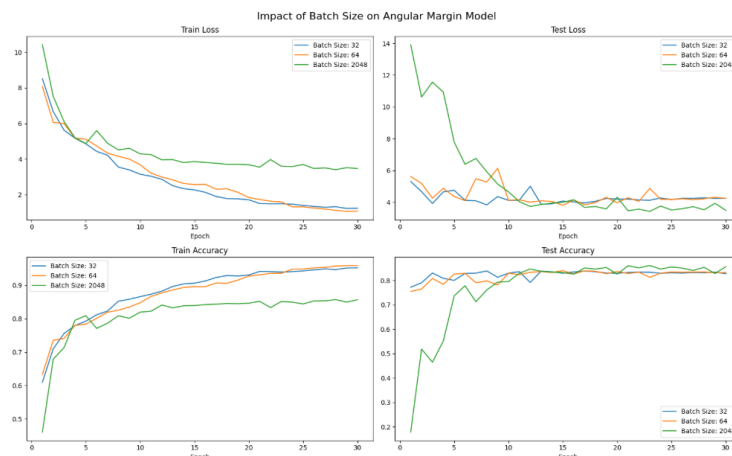
با جایگذاری این مقادیر در رابطه اصلی، به سادگی نتیجه می‌شود:

$$w^T x = (1)(1) \cos(\theta) = \cos(\theta) \quad (41.1)$$

بنابراین خروجی لایه قبل از تابع خطا (Logits)، دقیقاً برابر با کسینوس زاویه بین نمونه ورودی و مرکز کلاس آن خواهد بود. این نمایش زاویه‌ای برای تشخیص چهره بسیار مناسب‌تر است، زیرا طول بردار ویژگی (که ممکن است تحت تاثیر روشنایی تصویر تغییر کند) را نادیده گرفته و صرفاً روی جهت و زاویه که نشان‌دهنده هویت ذاتی است تمرکز می‌کند. ۲. تعریف Angular Margin Loss در این روش، برای افزایش واریانس بین کلاسی و دورتر کردن کلاس‌ها از یکدیگر، یک حاشیه زاویه‌ای ثابت با نام m از لاجیت مربوط به کلاس صحیح کسر می‌شود. فرمول استفاده شده در پیاده‌سازی به شکل زیر است:

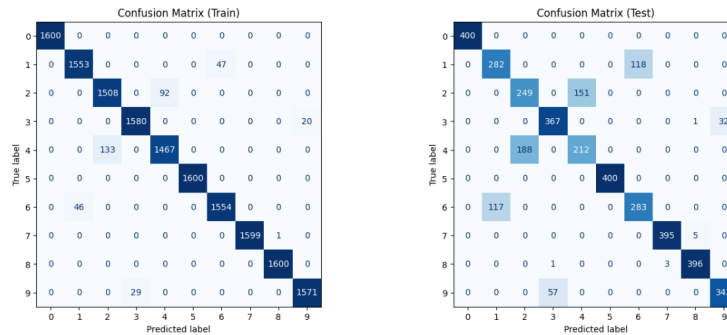
$$\hat{y} = \cos(\theta_y) - m \quad (42.1)$$

مقدار این حاشیه در بازه $[0.1, 0.5]$ قابل تنظیم است که در پیاده‌سازی ما مقدار 0.35 برای آن در نظر گرفته شد. پس از کسر حاشیه، مقادیر در یک ضریب مقیاس (Scale) ضرب شده (بخاطر توضیحاتی که در بخش تئوری دادم) و سپس تابع استاندارد CrossEntropy روی آن‌ها اعمال می‌گردد. ۳. آموزش مدل با Angular Margin Loss مدل با تابع هزینه جدید، بهینه‌ساز Adam (با نرخ یادگیری 0.0005 به دلیل بزرگتر بودن گرادین‌های این تابع نسبت به Softmax) و به مدت ۳۰ اپیاک آموزش داده شد. مانند بخش قبل، تاثیر اندازه‌های دسته ۳۲، ۶۴ و ۲۰۴۸ بررسی گردید.



شکل ۸.۱: نمودارهای تغییرات تابع زیان و دقت برای مجموعه‌های آموزش و تست در طول ۳۰ اپیاک با استفاده از تابع Angular Margin.

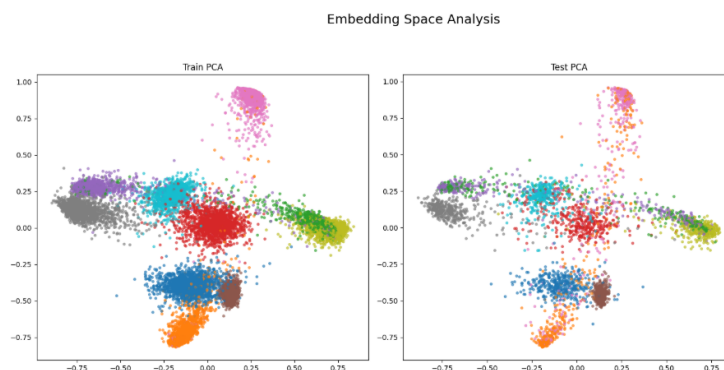
تحلیل تاثیر تغییر Batch Size: دسته $\text{Batch Size} = 32$: گرادین‌های تیز تابع حاشیه زاویه‌ای در ترکیب با دسته‌های کوچک، باعث پرش‌های ناگهانی در فضای بهینه‌سازی شده و نمودار خطای تست نوسانات زیادی را نشان می‌دهد. دسته $\text{Batch Size} = 64$: این حالت تعادل مناسبی ایجاد کرده و به دقت قابل قبولی روی مجموعه تست می‌رسد، هرچند که در میانه‌های راه همچنان نوساناتی در خطای تست دیده می‌شود. دسته $\text{Batch Size} = 2048$: این حالت در ایپاک‌های اولیه بسیار ضعیف عمل می‌کند (به دلیل تعداد کم به‌روزرسانی در هر ایپاک)، اما از اواسط آموزش پایدارترین مسیر همگرایی را طی کرده و در نهایت کمترین نوسان خطای تست را دارد. با این حال، به منظور حفظ شرایط عادلانه برای مقایسه با بخش قبل، مدل با Batch Size 64 به عنوان مدل نهایی برای استخراج متریک‌ها انتخاب شد. دقت نهایی این مدل روی داده‌های آموزش 95.94% و روی داده‌های تست 83.17% گزارش گردید. ۴. ارزیابی مدل و Confusion_Matrix برای ارزیابی دقیق، پیش‌بینی‌های مدل روی هر دو مجموعه رسم شد.



شکل ۹.۱: ماتریس درهم‌ریختگی مدل Angular Margin روی مجموعه‌های آموزش و تست.

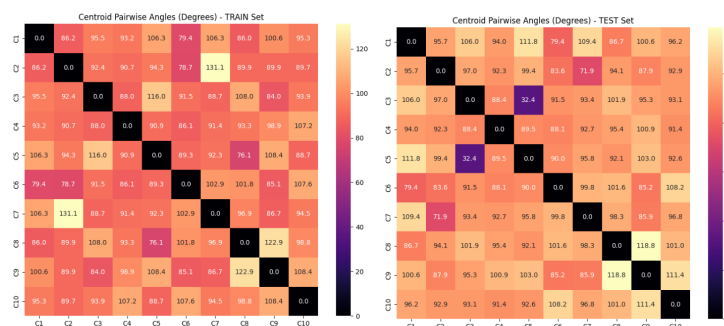
از Confusion_Matrix می‌فهمیم که مدل روی داده‌های آموزش به شدت فیت شده است (بیشتر مقادیر روی قطر اصلی نزدیک به ۱۶۰۰ هستند). با این حال، در داده‌های تست همچنان خطاهایی بین جفت کلاس‌های ذاتا نزدیک (مانند ۳ و ۵ یا ۲ و ۷) رخ می‌دهد. ماهیت تصادفی و نویزی بالای این داده‌ها (واریانس 1.5) باعث می‌شود که داده‌های مرزی کاملاً در هم تنیده باشند و خطای غیرقابل اجتنابی در تست بماند. ۵. تحلیل فضای Embedding مهم‌ترین بخش ارزیابی تابع هزینه زاویه‌ای، مشاهده توزیع داده‌ها در فضای ویژگی است. خروجی‌ها با استفاده از الگوریتم PCA استخراج و ترسیم شدند.

در مقایسه با حالت آموزش با Softmax که پرتوها حالت شعاعی داشتند، در اینجا داده‌ها روی سطح مقطع‌هایی کمان‌شکل توزیع شده‌اند. این شکل هندسی نتیجه مستقیم اعمال قید نرمال‌سازی L2 است که تمام بردارها را ملزم می‌کند روی سطح یک کره واحد قرار بگیرند. مقایسه فاصله مراکز کلاس‌ها: میانگین فاصله بین‌کلاسی در مدل Softmax برابر با 1.3557 بود، اما در این مدل به 1.4554 افزایش یافته است. این



شکل ۱۰.۱: نگاشت دوبعدی فضای ویژگی مدل Angular Margin.

افزایش فاصله تایید می‌کند که تابع Angular Margin با موفقیت توانسته است کلاس‌ها را از هم هل داده و مرزهای تصمیم‌بازتری ایجاد کند. به تبع آن، میانگین فاصله درون‌کلاسی نیز (از 0.1840 به 0.5040) افزایش یافته است، زیرا داده‌های نویزی برای فرار از کلاس‌های همسایه مجبور شده‌اند روی کمان‌های مربوط به کلاس خود کشیده شوند. ۶. تحلیل زاویه‌ای کلاس‌ها به منظور بررسی زوایای بین کلاس‌ها در فضای ویژگی، نقشه حرارتی (Heatmap) مربوطه رسم شد.



شکل ۱۱.۱: نقشه حرارتی زاویه به درجه بین مراکز کلاس‌ها در فضای ویژگی نرمال شده.

با مقایسه این نقشه حرارتی با مدل Softmax معمولی در می‌یابیم که زوایای بین کلاس‌ها به طور معناداری بزرگتر و یکنواخت‌تر شده‌اند و مقادیر بیشتر در محدوده ۹۰ تا ۱۱۰ درجه قرار گرفته‌اند. این افزایش فاصله زاویه‌ای دقیقاً همان دستاوردی است که مدل برای بهبود قدرت تفکیک‌پذیری در مسائل تشخیص چهره به دنبال آن بوده است و مقاومت مدل را در برابر داده‌های سخت افزایش می‌دهد.

۴.۲.۱ مقایسه نهایی، تحلیل و ارتباط با مقاله IAM

در این بخش نتایج به دست آمده از دو مدل آموزش داده شده جمع بندی شده و تفاوت های ساختاری آن ها تحلیل می گردد. همچنین مفاهیم تئوری مقاله IAM در قالب این نتایج عملی بررسی می شوند.

مقایسه کمی

جدول زیر مقادیر به دست آمده برای معیارهای مختلف را در دو مدل Softmax CNN و Angular Margin CNN (Embedding) برای بهترین اندازه دسته (۶۴) نشان می دهد. فواصل مستقیماً در فضای ویژگی (Embedding) محاسبه شده اند.

جدول ۱۰۱: مقایسه عملکرد و متریک های فضای ویژگی بین مدل پایه و مدل مبتنی بر حاشیه زاویه ای

معیار	Softmax CNN (Train)	Softmax CNN (Test)	Angular Margin CNN (Train)	Angular Margin CNN (Test)
Accuracy	89.83%	83.28%	95.94%	83.17%
Macro Precision	9102	8383	9771	8322
Macro Recall	9044	8328	9770	8318
Mean inter-class distance	1.3653	1.3557	1.4750	1.4554
Mean intra-class distance	0.1588	0.1840	0.2535	0.5040

تحلیل زاویه ای کلاس ها

مرکز هر کلاس در فضای ویژگی استخراج شده و زاویه بین این مراکز برای هر دو مدل محاسبه و رسم گردیده است (همان طور که این نقشه های حرارتی پیش تر در شکل های ۷.۱ و ۱۱.۱ در بخش های قبلی آورده شده اند). با مقایسه دو نقشه حرارتی مشخص می شود که مدل مبتنی بر حاشیه زاویه ای (Angular Margin) زوایای بسیار بزرگتر و منظم تری بین کلاس ها ایجاد کرده است. در مدل پایه، زاویه بین برخی کلاس ها بسیار کوچک است، اما در مدل زاویه ای، سیستم با موفقیت این زوایا را باز کرده و بیشتر آن ها را در محدوده ۹۰ تا ۱۱۰ درجه قرار داده است که نشان دهنده توزیع یکنواخت تر و با فاصله تر روی فضای کروی است.

تحلیل گرادیان و رفتار آموزشی

اول بیاین ریاضی طور بررسی کنیم، احتمال پیش بینی شده برای کلاس صحیح به این شکل در می آید:

$$p_y = \frac{e^{s(\cos(\theta_y) - m)}}{e^{s(\cos(\theta_y) - m)} + \sum_{j \neq y} e^{s \cos(\theta_j)}} \quad (43.1)$$

در بهینه سازی شبکه با تابع Cross-Entropy، اندازه گرادیانی که به سمت وزن ها و ویژگی ها پخش می یابد، ارتباط مستقیمی با مقدار $1 - p_y$ دارد.

(در صورت پروژه s را نداده بود ولی در بخش تئوری استدلال کردیم چرا مهمه و موقع پیاده سازی هم ۱ گذاشته شد و نتیجه خوبی داده نشد بخاطر همین ما هم فرم کلی ترش را در نظر میگیریم.)

حالا بیایید رفتار این کسر را برای نمونه‌های سخت بررسی کنیم. نمونه سخت یعنی داده‌ای که زاویه θ_y آن با مرکز کلاس خودش به نسبت باز است، در نتیجه $\cos(\theta_y)$ مقدار کوچکی دارد. وقتی ما مقدار ثابت حاشیه یعنی m را از این مقدار کوچک کم می‌کنیم، توان صورت کسر به شدت افت می‌کند. این افت باعث می‌شود مخرج کسر به خاطر حضور کلاس‌های رقیب، بسیار بزرگتر از صورت شود و در نتیجه p_y خیلی کم شود و یعنی عبارت $1 - p_y$ (که ضریب گرادیان ماست) خیلی زیاد شود و در نتیجه شبکه عصبی یک سیگنال خطای بسیار قوی (گرادیان بزرگ) برای نمونه سخت تولید می‌کند. در نقطه مقابل، برای نمونه‌های آسان زاویه θ_y بسیار کوچک و در نتیجه $\cos(\theta_y)$ نزدیک به ۱ است؛ پس حتی پس از کسر مقدار m ، توان کسر همچنان به اندازه کافی بالاست تا p_y مقدار مناسبی داشته باشد و گرادیان کمتری تولید کند.

حالا یک درک شهودی هم می‌توانیم داشته باشیم ازش، فضای ویژگی‌ها را به شکل یک کره در نظر بگیرید که هر کلاس روی این کره برای خودش یک منطقه دارد. در Softmax معمولی، مرز تصمیم‌گیری دقیقاً وسط زاویه بین دو کلاس است. اگر یک نمونه سخت دقیقاً روی این مرز باشد، شبکه به آن نمره مرزی می‌دهد و چون مرز را رد کرده، فشار زیادی برای هل دادن آن به سمت مرکز کلاس وارد نمی‌کند. رابطه $\cos(\theta_y) - m$ در واقع یک جریمه اجباری است. شبکه از امتیاز واقعی نمونه که $\cos(\theta_y)$ است، یک مقدار ثابت m کم می‌کند و بعداً می‌سنجد هنوز در کلاس درستش هست یا نه. برای نمونه‌های آسان که عمیقاً در منطقه کلاس خودشان هستند، کم شدن مقدار m باعث خروج آن‌ها از مرز نمی‌شود. اما برای نمونه‌های سخت که در حاشیه و نزدیک مرز کلاس‌های دیگر قرار دارند، وقتی m از نمره‌شان کسر می‌شود، آن‌ها به صورت مصنوعی به داخل منطقه کلاس اشتباه پرتاب می‌شوند. این اتفاق باعث می‌شود شبکه خطایش زیاد شود و بخواهد نمونه‌های سخت را از مرز دور کرده (با کمک گرادیان بزرگترش) و به سمت مرکز منطقه امن کلاس خودشان هل دهد تا دفعه بعد حتی با کسر جریمه m ، باز هم به کلاس دیگری متمایل نشوند. حالا روش IAM هم همینکار را میکند و هدفش این است که نمونه‌های سخت را از هم دور کند ولی این کار را به صورت Adaptive انجام میدهد. صورت کسر تابع IAM را به گونه‌ای طراحی کرده‌اند که نشان‌دهنده میانگین شباهت بین ویژگی x_i و وزن کلاس‌های اشتباه (w_j که $j \neq y_i$) باشد. اگر یک نمونه سخت باشد، یعنی زاویه آن با یکی از کلاس‌های اشتباه بسیار کوچک باشد، شباهت کسینوسی آن‌ها بزرگ می‌شود. این امر باعث می‌شود صورت کسر به شدت بزرگ شود و در نتیجه مقدار تابع هزینه IAM بالا برود یعنی تابع IAM به صورت Adaptive زوایای کوچکتر بین کلاسی (یعنی همان نمونه‌های سخت) را با جریمه سنگین‌تری مواجه می‌کند. خیلی پس شبیه هم هستند و بیشتر فرقی‌شان در اینه که در روش AM-Loss، این کار با اعمال یک جریمه ثابت روی لاجیت کلاس صحیح انجام می‌شود که به صورت غیرمستقیم نمونه‌های مرزی را تحت فشار قرار می‌دهد. در

روش IAM، این کار با هدف قرار دادن مستقیم کلاس‌های اشتباه انجام می‌شود؛ به طوری که هرچه یک داده به کلاس اشتباه نزدیک‌تر باشد (نمونه سخت‌تر)، تابع هزینه به گرادیان بزرگتری بین آن‌ها ایجاد می‌کند تا واریانس بین کلاسی را افزایش دهد. دقیقاً حدست درسته. ایده اصلی و اوج هنر این مقاله رسیدن به همین ترکیب از طریق منظم‌سازی (Regularization) است. نیازی نیست ما بین توابع هزینه مبتنی بر کلاس نهایی (مثل Softmax) یا توابع مبتنی بر زاویه انتخاب کنیم؛ میتوان تابع زاویه‌ای معرفی شده را به عنوان یک عبارت Regularization به توابع هزینه موجود اضافه کنیم تا نقاط ضعف آن‌ها پوشش داده شود.

این ایده در مقاله نیز آمده است به صورت زیر:

$$L = L_{base} + \beta L_{IAM} \quad (44.1)$$

(جای L_{IAM} میتوان مثلاً پیاده سازی خودمون رو گذاشت و L_{base} هم مثلاً همون Softmax باشه).

تحلیل نهایی

۱. آیا بهبود دقت الزاماً زیاد بوده است؟ چرا؟

همان‌طور که نتایج نشان می‌دهد، دقت تقریباً یکسان درآمده است (دقت حدود ۸۳ درصد برای هر دو مدل) این بخاطر ماهیت داده‌های مصنوعی هست که درست کردیم. داده‌ها از توزیع نرمال با واریانس مشخصی تولید شده‌اند که باعث ایجاد هم‌پوشانی ذاتی بین کلاس‌ها در فضای ویژگی اصلی می‌شود. داده‌هایی که ذاتاً از نظر آماری در فضای یکدیگر نفوذ کرده‌اند، احتمالاً هیچ مرز تصمیمی برایش وجود نداشته باشد که کاملاً بتوانند جدا کنند و خطای مینیمی همواره خواهد داشت. تابع هزینه مبتنی بر حاشیه زاویه‌ای توزیع داده‌ها را در فضای نهان بهبود می‌بخشد، اما نمی‌تواند هم‌پوشانی‌های آماری اولیه را به طور کامل خنثی کند.

۲. چرا حتی با دقت مشابه، فضای ویژگی مدل مبتنی بر حاشیه بهتر است؟

در ماتریس زوایای مدل پایه، مشاهده می‌شود که فاصله زاویه‌ای برخی کلاس‌ها به شدت در هم فرو ریخته است؛ به عنوان مثال زاویه بین کلاس ۳ و ۵ در داده‌های تست تنها ۸.۵ درجه و زاویه کلاس ۴ و ۱۰ حدود ۳۱.۵ درجه است. تابع Softmax استاندارد صرفاً به دنبال یافتن یک مرز تصمیم برای جدا کردن کلاس‌ها است و به محض اینکه داده از مرز عبور کند، دیگر گرادیان قابل توجهی برای دور کردن مراکز کلاس‌ها تولید نمی‌کند. در مقابل، در ماتریس زوایای مربوط به AM-Loss، همان زاویه بحرانی ۸.۵ درجه‌ای بین کلاس ۳ و ۵ به ۳۲.۴ درجه بهبود یافته است و سایر مقادیر خارج از قطر اصلی نیز به طور کلی مقادیر بالاتری دارند. در نمودار PCA نیز توزیع شعاعی کلاس‌ها بسیار ساختاریافته‌تر است و نشان می‌دهد که کلاس‌ها در سطح فضای کروی

به شکل یکنواخت‌تری پخش شده‌اند. این یعنی مدل مبتنی بر حاشیه، یک فضای نمایش بسیار مقاوم‌تر و با قابلیت تفکیک بالاتر ساخته است.

۳. اگر این داده ساده‌تر نبود کدام روش برتری بیشتری داشت؟

قطعا روش‌های مبتنی بر حاشیه، دو ضعف اساسی تابع Softmax در مواجهه با داده‌های پیچیده، نویزی و دارای هم‌پوشانی (مانند تصاویر واقعی چهره در شرایط نوری مختلف) وجود دارد: اولین مورد، مسئله تقلب شبکه با استفاده از اندازه بردار ویژگی است. در تابع Softmax استاندارد، خروجی لاجیت بر اساس ضرب داخلی محاسبه می‌شود، یعنی $w^T x = ||w|| ||x|| \cos(\theta)$. در مواجهه با داده‌های پیچیده، شبکه عصبی تمایل دارد مسیر راحت‌تر را انتخاب کند؛ به این معنا که برای نمونه‌های آسان و باکیفیت، اندازه بردار ویژگی یعنی $||x||$ را به شدت بزرگ می‌کند تا مقدار تابع هزینه سریع‌تر کاهش یابد. این در حالی است که بزرگی اندازه بردار هیچ ارتباط معنایی با هویت شخص ندارد و تفکیک‌پذیری واقعی را بهبود نمی‌دهد. روش‌های مبتنی بر حاشیه با اعمال نرمال‌سازی L2 روی ویژگی‌ها و وزن‌ها، این متغیر مزاحم را حذف کرده و اندازه را به یک مقیاس ثابت s محدود می‌کنند. با این کار، شبکه مجبور می‌شود برای کاهش خطا، منحصرأ روی بهینه‌سازی زاویه θ که نمایانگر واقعی تمایز هویت‌هاست، تمرکز کند. دومین مورد هم این هست که در شرایطی که داده‌ها تنوع بالایی دارند، مرزهای تصمیم‌گیری Softmax به شدت درهم‌تنیده و شکننده می‌شوند. این مدل به سرعت روی نویزهای داده‌های آموزشی دچار overfit شده و در مواجهه با داده‌های تست شکست می‌خورد. اما روش‌های مبتنی بر حاشیه، علاوه بر ایزوله کردن زاویه، با کسر یک مقدار ثابت m ، یک منطقه خالی اجباری بین کلاس‌ها ایجاد می‌کنند. این حاشیه امن از تداخل داده‌های پیچیده جلوگیری کرده و قدرت Generalization مدل را روی داده‌های جدید به شدت بالا می‌برد.

۴. چرا مقاله IAM این روش را به عنوان یک منظم‌ساز معرفی می‌کند؟

تابع هزینه IAM به صورت Adaptive روی دفع کلاس‌های نزدیک به هم تمرکز دارد. اگر شبکه از ابتدا و تنها با این تابع آموزش ببیند، ممکن است در مراحل اولیه آموزش دچار ناپایداری شود (مثلا در مقاله برای L_{sphere} گفته شده که به تنهایی نمیتواند همگرا بشود و به عنوان یک بهینه‌ساز برای Softmax بکار میرود). یا نتواند خوشه‌های اولیه را به درستی شکل دهد. با معرفی IAM به عنوان یک Regularizer در کنار یک تابع هزینه پایه، مدل ابتدا از پایداری و قدرت دسته‌بندی تابع هزینه اصلی بهره می‌برد. سپس، تابع IAM وارد عمل شده و بهینه‌سازی فضای ویژگی را هدایت می‌کند تا زوایای کوچک بین کلاسی را جریمه و فواصل را بیشینه کند.

۵. مدل آموزش دیده بر حسب حاشیه در چه کارکردی ممکن است موثرتر و مفیدتر باشد؟

زمانی که ما در محیط تست با کلاس‌ها جدیدی روبه‌رو می‌شویم که شبکه در طول آموزش هرگز آن‌ها را ندیده است، لایه طبقه بند نهایی دیگه فایده ای برای ما ندارد و ما صرفاً بردار ویژگی استخراج شده را با استفاده از معیار شباهت کسینوسی نسبت به یک پایگاه داده می‌سنجیم. بنابراین، داشتن یک فضای نهان که فواصل درون‌کلاسی حداقل و فواصل بین‌کلاسی حداکثر هستند، شرط اساسی برای کارکرد صحیح این سیستم‌ها است (اشتباه نمیگه یک کلاس دیگه هست چون دوره به نسبت از همه چی در فضای نهان). مثال هایی از وقتایی که این سناریو ها ممکن است رخ بدهد میشه مثلاً Face ID و سیستم سرچ تصویر را نام برد.

کتاب نامه

- [1] Sun, Jingna, Yang, Wenming, Gao, Riqiang, Xue, Jing-Hao, and Liao, Qing-min. Inter-class angular margin loss for face recognition.